

Lecture 7: Outline

- ▶ Matrix decompositions
 - ▶ using eigenvalues and eigenvectors.
- ▶ A generalization for non-square matrices
- ▶ Application for compressing.

Lecture 7: A simple decomposition

- ▶ Eigenvalue decomposition for A .
- ▶ Conditions for the existence:
 - ▶ $A \in \mathbb{R}^{n \times n}$ – square-matrix.
 - ▶ **Symmetry**.
- ▶ The general shape then: $A = QDQ^{-1}$.
 - ▶ Q composed on the different eigenvectors (of length *one*).
 - ▶ They are **orthogonal to each other**; Q is called orthogonal.
 - ▶ Therefore, $Q^{-1} = Q^T$ (we do not have to compute inverse).
 - ▶ $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ is diagonal composed by the eigenvalues of A .

Lecture 7: A simple decomposition - explanation, remarks

First advantage: A clear interpretation how does A act to a vector $\mathbf{v}$:

- ▶ In a few words: in an appropriate basis, A is simply the multiplication with a diagonal matrix.
 - ▶ The appropriate one is constituted by the columns of Q .
- ▶ In a bit more details: Rewrite $\mathbf{v}$ with new coordinates in the basis constituted of eigenvectors
 - ▶ to get $Q^{-1}\mathbf{v}$.
- ▶ Apply the linear transformation.

In the new basis, this is simply a multiplication with the diagonal matrix D to get

- ▶ $DQ^{-1}\mathbf{v} = [\lambda_1 \cdot Q^{-1}\mathbf{v}[1], \lambda_2 \cdot Q^{-1}\mathbf{v}[2], \dots, \lambda_n \cdot Q^{-1}\mathbf{v}[n]].$
- ▶ Transform it back into the original coordinates:
 - ▶ to get $QDQ^{-1}\mathbf{v} = A\mathbf{v}.$

Lecture 7: A simple decomposition - explanation, remarks

Second advantage: Enables fast computation of matrix functions.

- ▶ One can define, e.g. the exponential of the (symmetric) matrix A and compute in this way:

$$e^A = Q \cdot \text{diag}(e^{\lambda_1}, e^{\lambda_2}, \dots, e^{\lambda_n}) \cdot Q^{-1}.$$

- ▶ Application: computing centrality of a node in a network.

www.math.kent.edu/~reichel/publications/matexp.pdf

A large disadvantage: Computationally hard/expensive to construct

- ▶ a full version of it;
- ▶ i.e. find all of the eigenvalues and eigenvectors.

Lecture 7: A more general case

- ▶ In general:
 - ▶ we have neither symmetric,
 - ▶ nor square matrices.
- ▶ Therefore, instead of the shape $A = QDQ^T$
 - ▶ different matrices should be on the right and left,
 - ▶ non-square matrix in the middle.
- ▶ OK, but we try to see the transformation $A : \mathbb{R}^n \rightarrow \mathbb{R}^k$ in a simple form:
 - ▶ First transformation into a new basis in $\mathbb{R}^n$.
 - ▶ Then multiplication of the components with given numbers (non-square matrix have no “eigenvalues”) and extending / projecting them into $\mathbb{R}^k$.
 - ▶ Transformation back with *another basis* in $\mathbb{R}^k$.

Lecture 7: Singular value decomposition (SVD) in details

- ▶ Given *any* matrix $A \in \mathbb{R}^{k \times n}$ it can be decomposed as:

$$A = U\Sigma V^T, \quad \text{where}$$

- ▶ $V \in \mathbb{R}^{n \times n}$ is *orthogonal*, i.e. its columns are orthogonal.
- ▶ $\Sigma \in \mathbb{R}^{k \times n}$ is a "*diagonal*" matrix
 - ▶ only "diagonal" elements $\Sigma_{j,j} \in \mathbb{R}$ can be non-zero,
 - ▶ they are (or can be assumed to be) non-negative,
 - ▶ they are sorted decreasingly.
 - ▶ Their name: singular values.
- ▶ $U \in \mathbb{R}^{k \times k}$ is *orthogonal*, i.e. its columns are orthogonal.
- ▶ Uniqueness is only for Σ .

Lecture 7: SVD decomposition - natural reduction

- ▶ Observation: if $n \neq k$ then indeed,
 $\text{rank } A \leq \min\{n, k\} := m.$
- ▶ The number of singular values is equal to m . Thus, Σ contains some zero rows.
- ▶ We drop out these and the dimensions become:
 $V^T \in \mathbb{R}^{m \times n}, \Sigma \in \mathbb{R}^{m \times m}, U \in \mathbb{R}^{k \times m}.$
- ▶ This is called the compact or reduced SVD. Python command for this: `np.linalg.svd(a, full_matrices=False)`
- In general: Σ is stored only as a vector.

Lecture 7: Singular value decomposition (SVD) - construction (may be skipped)

- ▶ Analytic construction
 - ▶ depending on the simple decomposition,
 - ▶ performed with formulas.

- ▶ If we have the desired form, take

$$A^T A = V \Sigma U^T U \Sigma V^T = V \Sigma \Sigma V^T.$$

- ▶ Decompose it:

$$V \Sigma \Sigma V^T = Q D Q^{-1}.$$

- ▶ Observe: $\Sigma^2 = D$ (take square root elementwise) and $V = Q$.
- ▶ Finally: $U \Sigma V^T = A \Rightarrow U = A V \cdot \Sigma^{-1}$.

Lecture 7: Singular value decomposition: application to compressing

- ▶ Given a matrix $A \in \mathbb{R}^{1920 \times 1080}$ corresponding to an image.
- ▶ We compute its reduced SVD decomposition as

$$A = U\Sigma V^T, \quad \text{where}$$

- ▶ $V \in \mathbb{R}^{1080 \times 1080}$, $\Sigma \in \mathbb{R}^{1080 \times 1080}$, $U \in \mathbb{R}^{1920 \times 1080}$.
- ▶ Instead keep the following:
 - ▶ the first 50 columns of U
 - ▶ the first 50 eigenvalues of Σ
 - ▶ the first 50 rows of V^T .

Lecture 7: Compressing using SVD - practical issues

- ▶ Originally: 1920×1080 digits.
- ▶ Using this kind of compressing:
 - ▶ for V^T : 50×1080 ,
 - ▶ for Σ : 50 - just take diagonal elements,
 - ▶ for U : 1920×50 .
 - ▶ Altogether: $\frac{3001 \cdot 50}{1080 \cdot 1920} \approx 0.072$
- ▶ If we can keep the quality of the picture: this is fine, but
 - ▶ it is rather expensive: never perform “full” SVD decomposition.
- ▶ Indeed, we just need to compute the “first” 50 singular values and corresponding vectors in U and V .

Lecture 8: Outline

- ▶ Abstraction of distances.
 - ▶ norm of vectors
 - ▶ norm of matrices.

Lecture 8: Back to the basics: vectors

- ▶ We know the conventional length in $\mathbb{R}^d$:

$$|\mathbf{v}| = \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2}.$$

- ▶ this is zero only for zero vector $\mathbf{0} \in \mathbb{R}^d$
- ▶ of course, for any $\lambda \in \mathbb{R}$: $|\lambda \mathbf{v}| = |\lambda| |\mathbf{v}|$,
- ▶ satisfies the triangle inequality: $|\mathbf{u} + \mathbf{v}| \leq |\mathbf{u}| + |\mathbf{v}|$.
 - ▶ A bit of work to verify it.
- ▶ Also, we can define the *distance* of $\mathbf{u}$ and $\mathbf{v}$ in $\mathbb{R}^d$ as:
$$d(\mathbf{u}, \mathbf{v}) = |\mathbf{u} - \mathbf{v}|.$$
- ▶ We intend to generalize this length and distance.

Lecture 8: Abstraction of length: the norm

- ▶ This can be defined on vector spaces (for us, $\mathbb{R}^d$ is sufficient).
 - ▶ Norm is defined as a function $\|\cdot\| : \mathbb{R}^d \rightarrow [0, \infty)$ which satisfies:
 - ▶ Norm is zero only for zero vector: $\|\mathbf{v}\| = 0 \Rightarrow \mathbf{v} = \mathbf{0}$.
 - ▶ For any $\lambda \in \mathbb{R}$ and $\mathbf{v} \in \mathbb{R}^d$: $\|\lambda\mathbf{v}\| = |\lambda|\|\mathbf{v}\|$.
 - ▶ Satisfies the triangle inequality: $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ for all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$.
- ▶ New (simple) examples:
 - ▶ $\|\mathbf{v}\|_\infty := \max_j |v_j|$,
 - ▶ $\|\mathbf{v}\|_1 := |v_1| + |v_2| + \cdots + |v_d|$,
 - ▶ $\|\mathbf{v}\|_2 := |\mathbf{v}|$ - the original length.

Lecture 8: Norms from another viewpoint

- ▶ In many real-life applications norm corresponds to energy.
 - ▶ Vector spaces can model a physical system. Vectors = states of the system.
 - ▶ Norm of a vector $\sim$ energy of the corresponding state.
- ▶ Example:
 - ▶ State: vector $\mathbf{v}$ - height of a water wave at fixed (equidistributed) points,
 - ▶ a norm of this $\sum_j |v_j|$ corresponds to the potential energy of the corresponding state.

Lecture 8: Abstraction of distance: the metric

- ▶ This can be defined on any set X (not only a space of vectors).
 - ▶ Metric is defined as a function $\rho : X \times X \rightarrow [0, \infty)$ which satisfies:
 - ▶ Only zero between the same elements:
 $\rho(x_1, x_2) = 0 \Rightarrow x_1 = x_2$.
 - ▶ Symmetric: $\rho(x_1, x_2) = \rho(x_2, x_1)$ for all $x_1, x_2 \in X$.
 - ▶ Satisfies the triangle inequality:
 $\rho(x_1, x_3) \leq \rho(x_1, x_2) + \rho(x_2, x_3)$ for all $x_1, x_2, x_3 \in X$.
- ▶ The pair (X, ρ) is called a metric space.

Lecture 8: Metrics - simple examples

Most general example:

- ▶ If $\| \cdot \|$ is a norm on the vector space X :
- ▶ The function $\rho(x_1, x_2) := \|x_1 - x_2\|$ will be a metric.

Further easy examples:

- $X =$ settlements on Hungary's Google map
 - (i) $\rho_1(x_1, x_2)$ = distance of x_1 and x_2 by car
 - (ii) $\rho_2(x_1, x_2)$ = distance of x_1 and x_2 by public transport
 - (iii) $\rho_3(x_1, x_2)$ = time to travel from x_1 to x_2 by car (at a given time)
- ▶ ρ_1 and ρ_2 are metrics,
- ▶ ρ_3 may not be symmetric (jam in one direction).

Lecture 8: Metrics - a real-life example

- ▶ Genetic distance; a particular case.
- ▶ Defined between individuals represented with gene sequences on all chromosomes.
- ▶ Mathematically: between vectors of given length,
 - ▶ components are integer vectors.
- ▶ Be careful: this is not $\mathbb{R}^d$, we do not have vector structure.
- A possible distance: number locations with different elements (genes).
- ▶ An interpretation: the minimal number of locations to change to transform one to the other.
 - ▶ Not realistic: if all gene sequences differ in one digits from the one of the other individual, this will be large.

Lecture 8: Metrics - genetic distance (continued)

- Another possibility: Nei's standard distance.
 - ▶ K - total length.
 - ▶ $u_{k,1}, u_{k,2}, \dots, u_{k,s_k}$ - the integer vector at component k .
- ▶ The distance is the following:

$$\rho(\mathbf{u}, \mathbf{v}) = -\ln \frac{\sum_{k=1}^K \sum_{j=1}^{s_k} u_{k,j} v_{k,j}}{\sqrt{\sum_{k=1}^K \sum_{j=1}^{s_k} u_{k,j}^2 \sum_{k=1}^K \sum_{j=1}^{s_k} v_{k,j}^2}}$$

- ▶ May seem complex - but the basic, realistic one.
 - ▶ Quite a bit calculation for triangle inequality.
- ▶ There is also a meaningful distance between sequences of different length.

Lecture 8: Metrics - in neural networks

- ▶ X set of all possible outputs
 - ▶ ρ is a meaningful metric on X ,
 - ▶ $x \in X$ is the real output,
 - ▶ $\hat{x} \in X$ is the prediction of x .
- ▶ $\rho(\hat{x}, x)$ is a measure of the quality of the approximation (prediction).
- ▶ In practice take a number pairs $(\hat{x}, x)$ and check the average of the values of $\rho(\hat{x}, x)$.

Lecture 8: Metrics - use proper scaling: motivation

An example:

- ▶ We have the following real outputs and predictions:

$$x_1 = [1, 2, 3, 4], \quad \hat{x}_1 = [1.1, 1.9, 3.1, 3.9],$$

$$x_2 = [1, 2, 3, 4, 5, 6, 7, 8], \quad \hat{x}_2 = [1.1, 1.9, 3.1, 3.9, 5.1, 5.9, 7.1, 7.9]$$

- ▶ Someone applies the $\|\cdot\|_1$ norm for measuring the reliability.
- ▶ In the first case: $\|x_1 - \hat{x}_1\|_1 = 0.4$,
while in the second: $\|x_2 - \hat{x}_2\|_1 = 0.8$.
- ▶ But they are of the same quality.
- ▶ To get a realistic norm (?), one should scale with the length of the vector (in this case)

Lecture 8: Metrics - use proper scaling: practice

Throughout: $\mathbf{x}$ -true value, $\hat{\mathbf{x}}$ -prediction, estimation,...

- ▶ Can not be given as "norm" - norms are defined on vector spaces
 - ▶ Vectors with different length cannot be added.
- ▶ Take the average distance:

$$d(\mathbf{x}, \hat{\mathbf{x}}) := \frac{1}{\text{length}(\mathbf{x})} \cdot \|\mathbf{x} - \hat{\mathbf{x}}\|_1$$

- ▶ Name: *mean absolute error*.
- ▶ Or similarly (mind the difference):

$$d(\mathbf{x}, \hat{\mathbf{x}}) := \sqrt{\frac{(x_1 - \hat{x}_1)^2 + \dots + (x_N - \hat{x}_N)^2}{N}} = \frac{1}{\sqrt{\text{length}(\mathbf{x})}} \cdot \|\mathbf{x} - \hat{\mathbf{x}}\|_2$$

- ▶ Name: *root mean squared error*.

Lecture 8: “Metrics” - examples (continued)

Motivation: often, components of $\mathbf{x}$ are given digits (integers).

- $\hat{\mathbf{x}}$ is of the same type
- ▶ Proportion of identical components:

$$d_*(\mathbf{x}, \hat{\mathbf{x}}) := \frac{1}{\text{length}(\mathbf{x})} \cdot \sum_{j: x_j = \hat{x}_j} 1.$$

- ▶ Name for arbitrary digits: *accuracy*.
- ▶ Name for 0-1 digits: *binary accuracy*.
- d_* is not any more a metrics in the math sense.
 - ▶ This measures *accuracy* in a 0 – 1 scale.
 - ▶ $d = 1 - d_*$ is already a metrics.
 - ▶ This can take values also in $[0, 1]$.

Lecture 8: “Metrics” - examples (continued)

- ▶ Sometimes, we use “one hot encoding”.
 - ▶ Example: 5 possible digits, digit “4” $\sim [0, 0, 0, 10]$
 - ▶ This is one component of $\mathbf{x}$.
 - ▶ shape of $\mathbf{x}$: $[[\dots], [\dots], \dots, [\dots]]$ - 2D array (matrix)
 - ▶ If we have a number of data: we have a 3D array.
 - ▶ Accordingly, one component of $\hat{\mathbf{x}}$:
Example: $[0.1, 0.15, 0.4, 0.92, 0.34]$.
 - ▶ Accordingly, we round up them:
$$\hat{\mathbf{x}} = \text{round}\{\hat{\mathbf{x}}\} = [0, 0, 0, 1, 0].$$
- ▶ We apply them d_* between $\mathbf{x}$ and $\hat{\mathbf{x}}$
 - ▶ Name: *categorical accuracy*.

Lecture 8: Metrics - weighting

- ▶ In many cases, some components are of great importance.
 - ▶ Example: in face detection, location of eyes is essential.
 - ▶ In the daily order of a shop, amount of fresh goods.
- ▶ Accordingly, in the metrics, components can be weighted with *positive* weights (normally, their sum is *one*).
- ▶ In any meaningful case, this does not harm metrics properties.
- ▶ Example:
 - ▶ $\mathbf{w} = [0.6, 0.2, 0.1, 0.1]$, $\mathbf{x} = [0, 0, 0, 1]$, $\hat{\mathbf{x}} = [1, 1, 0, 1]$
 - ▶ $\text{binary accuracy}_{\mathbf{w}}(\mathbf{x}, \hat{\mathbf{x}}) = 0.1 \cdot 1 + 0.1 \cdot 1 = 0.2$.
 - ▶ Low accuracy; we did not catch substantial components.
- For an exhausting list for all possibilities, see
www.tensorflow.org/api_docs/python/tf/keras/metrics